

PYTHON UNIT 4 QB

(2 – marks)

1. What is input function in Python?

1. The **input() function** in Python is used to take input from the user through the keyboard. It allows the program to receive data while the program is running.
2. The input() function always returns the entered value as a **string by default**. It can be converted to other types like integer or float using type casting.
3. A message or prompt can be displayed to guide the user about what to enter. The program waits until the user enters a value and presses Enter.
4. The entered value is stored in a variable and used in the program. This helps make programs interactive and user-friendly.

2. Define output formatting.

1. **Output formatting** means displaying output in a clear and organized way. It helps make program results easier to read and understand.
2. Python provides methods like **comma separation, format() method, and f-strings** for formatting output. These methods allow text and variables to be displayed together.
3. Output formatting can also control number display such as **decimal places and alignment**. This makes the output more structured.
4. Proper formatting improves readability and presentation of results. It makes program output look neat and professional.

3. What is the use of print() function?

1. The **print() function** in Python is used to display output on the screen. It can show text, variables, and results of a program.
2. It helps programmers communicate information or results to the user. It is commonly used to display messages.
3. The print() function can display **multiple values together** using commas. Python automatically adds spaces between the values.
4. It is also used with formatting methods like **format() and f-strings** to display output in a structured form.

4. What is a file in Python?

1. A **file in Python** is used to store data permanently on a storage device. It allows programs to save and retrieve information later.
2. Files can store different types of data such as **text, images, audio, or videos**. This helps manage large amounts of data.
3. Python provides functions like **open(), read(), write(), and close()** to perform file operations. These functions help read and write data.
4. Files are mainly of **two types: text files and binary files**. Text files store readable data while binary files store non-text data.

5. Name different file modes in Python.

1. Python provides different **file modes** that define how a file will be used. These modes specify whether the file will be read, written, or appended.
2. The **r mode** is used to open a file for reading data. If the file does not exist, Python raises a `FileNotFoundError`.
3. The **w mode** is used to open a file for writing data. If the file already contains data, it will be overwritten.
4. The **a mode** is used to append data at the end of a file. If the file does not exist, Python creates a new file.

6. What is the difference between text file and binary file?

1. A **text file** stores data in readable characters. A **binary file** stores data in binary form (0s and 1s).
2. Text files can be opened and read easily using a text editor. Binary files usually appear unreadable to humans.
3. Text files are used to store documents, logs, or program data. Binary files are used for images, audio, videos, and other non-text data.
4. In Python, text files use modes like **r, w, and a**, while binary files use **rb, wb, and ab** modes.

7. What is CSV file?

1. A **CSV (Comma Separated Values) file** is a format used to store tabular data. The values in the file are separated by commas.
2. Each line in a CSV file represents a **row of data**. Each value separated by a comma represents a column.
3. CSV files are widely used to exchange data between applications like spreadsheets and databases.
4. Python provides a **csv module** that helps in reading and writing CSV files easily.

8. What is the use of open() function?

1. The **open() function** in Python is used to open a file before performing operations on it. It allows the program to access the file for reading or writing.
2. The function requires two main arguments: **file name and mode**. The mode specifies how the file will be used.
3. When a file is opened, Python returns a **file object**. This object is used to perform operations like `read()` and `write()`.
4. After completing file operations, the file should be closed using **close()**. This releases system resources and ensures proper file handling.

(4 – marks)

1. Explain different file opening modes in Python with examples.

1. In Python, files are opened using the **open() function**, which requires the file name and mode as arguments. The mode specifies how the file will be used, such as reading, writing, or appending data.
2. The **read mode (r)** is used to open a file for reading its contents. If the file does not exist, Python raises an error. Example: `file = open("data.txt","r")`.
3. The **write mode (w)** is used to write data into a file. If the file already exists, its previous contents are erased and new data is written. Example: `file = open("data.txt","w")`.
4. The **append mode (a)** is used to add new data at the end of an existing file without deleting previous content. Example: `file = open("data.txt","a")`.
5. The **read and write modes (r+, w+, a+)** allow both reading and writing operations in a file. These modes are useful when a program needs to update file contents as well as read existing data.

2. Write a program to read a text file and display its contents.

1. In Python, a text file can be read using the **open() function in read mode (r)**. The program reads the file contents and displays them on the screen.
2. After opening the file, the **read() method** is used to read all the data stored in the file. This method returns the entire content as a string.
3. Example program to read a file:

```
file = open("data.txt", "r")
content = file.read()
print(content)
file.close()
```

4. In this program, the file named **data.txt** is opened in read mode. The `read()` function retrieves the content and the `print` statement displays it on the screen.
5. After reading the file, the **close() method** is used to close the file. Closing the file releases system resources and ensures safe file handling.

3. Explain formatted output using format() method with example.

1. In Python, formatted output is used to display values in a structured and readable format. The **format() method** allows inserting variables into a string in a flexible way.
2. The format() method uses placeholders {} inside a string where values will be inserted. These placeholders are replaced by the values provided inside the format() function.
3. Example of formatted output:

```
name = "Rahul"  
age = 20  
print("My name is {} and I am {} years old".format(name, age))
```

4. In this program, the placeholders {} are replaced by the values of **name** and **age**. This method helps organize output clearly and makes the program easier to read.
5. The format() method also allows specifying positions or formatting numbers such as decimal places. For example: print("Value is {:.2f}".format(3.4567)) displays the value up to two decimal places.

4. A program needs to store student names entered by user into a file. Write code for writing input data into a file.

1. Python allows writing user input into files using the **write() method**. This method stores the given data into a file opened in write or append mode.
2. The program first opens a file using the **open() function in write mode (w)**. Write mode creates the file if it does not exist or overwrites the existing content.
3. Example program to store student names:

```
file = open("students.txt", "w")  
  
name = input("Enter student name: ")  
file.write(name)  
  
file.close()
```

4. In this program, the user enters the student name through the input function. The write() method stores the entered name in the file **students.txt**.
5. After writing the data, the **close() method** is used to close the file. Closing the file ensures that the data is saved properly and system resources are released.

5. Explain reading and writing binary files with example.

1. Binary files store data in binary format instead of plain text. These files are commonly used for images, audio, videos, or compiled programs where data is stored as bytes.
2. In Python, binary files are opened using **binary modes such as rb (read binary) and wb (write binary)**. The data is handled as bytes rather than normal text characters.
3. Writing to a binary file means storing data in byte format using the **write() method**. This allows programs to save binary information such as images or serialized data.
4. Example program for writing and reading binary file:

```
# Writing binary data
file = open("data.bin", "wb")
file.write(b"Hello World")
file.close()
```

```
# Reading binary data
file = open("data.bin", "rb")
content = file.read()
print(content)
file.close()
```

6. Your program should append new data to an existing file without deleting old data. Write code.

1. In Python, the **append mode (a)** is used to add new data to an existing file. This mode ensures that previously stored data remains unchanged.
2. When a file is opened in append mode, the file pointer automatically moves to the end of the file. Any new data written will be added after the existing content.
3. The program uses the **write() method** to add the new information into the file. This method stores the text provided by the user.
4. Example program to append data to a file:

```
file = open("data.txt", "a")

text = input("Enter text to append: ")
file.write(text + "\n")

file.close()
```

7. Explain CSV file handling using Python.

1. A **CSV (Comma Separated Values) file** is a file format used to store tabular data such as spreadsheets or database records. Each line represents a row and values are separated by commas.
2. Python provides a built-in **csv module** to read and write CSV files easily. This module simplifies handling structured data stored in rows and columns.
3. To read a CSV file, the program opens the file and uses the **csv.reader() function**. This function reads each row of the CSV file as a list.
4. Example program to read CSV file:

```
import csv
```

```
file = open("students.csv", "r")  
reader = csv.reader(file)
```

```
for row in reader:  
    print(row)
```

```
file.close()
```

5. CSV file handling is useful for storing and processing large datasets in Python programs. It is commonly used in data analysis, reporting systems, and database exports.

(5 – marks)

1. Explain input and output formatting techniques in Python with examples.

1. In Python, **input functions** are used to take data from the user during program execution. The most common input function is **input()**, which reads user input as a string.
2. The input() function can be used with type conversion to accept different data types. For example:
age = int(input("Enter age: ")) converts the entered value into an integer.
3. **Output formatting** is used to display output in a neat and readable form. Python provides methods such as comma separation, **format() method**, and **f-strings** for formatting output.
4. In the **format() method**, placeholders are used inside the string to display variable values. Example:
print("Name: {} Age: {}".format(name, age)).
5. **f-strings** are a modern and recommended way to format output in Python. Example:
print(f"Name: {name} Age: {age}").

2. Write a program to copy contents from one file to another.

1. Python allows copying data from one file to another using file handling functions. First, the source file is opened in **read mode** and the destination file is opened in **write mode**.
2. The data from the source file is read using the **read() function**. This function reads all the content of the file.
3. After reading the data, it is written into another file using the **write() function**. This copies the content into the new file.
4. Both files should be closed after completing the operations. Closing files helps free system resources and ensures proper file handling.

```
f1 = open("source.txt", "r")  
f2 = open("destination.txt", "w")
```

```
data = f1.read()  
f2.write(data)
```

```
print("File copied successfully")
```

```
f1.close()  
f2.close()
```


3. Develop a program to store employee details (ID, Name, Salary) into a CSV file and display them.

1. A **CSV (Comma Separated Values) file** is used to store tabular data where values are separated by commas. Python provides a built-in **csv module** to easily read and write CSV files.
2. First, the CSV file is opened in **write mode** using the `open()` function with the `newline` attribute. Then the **`csv.writer()`** function is used to create a writer object.
3. The employee details such as **ID, Name, and Salary** are taken as input from the user. These values are written into the CSV file using the **`writerow()`** function.
4. After storing the data, the CSV file is opened again in **read mode**. The **`csv.reader()`** function reads the stored data from the file.
5. The program then loops through each row of the CSV file and displays the employee details on the screen. This helps verify that the data has been stored correctly.

```
import csv
```

```
with open("employee.csv", "w", newline="") as f:
```

```
    w = csv.writer(f)
```

```
    w.writerow(["ID", "Name", "Salary"])
```

```
    n = int(input("Enter number of employees: "))
```

```
    for i in range(n):
```

```
        eid = input("Enter ID: ")
```

```
        name = input("Enter Name: ")
```

```
        salary = input("Enter Salary: ")
```

```
        w.writerow([eid, name, salary])
```

```
print("Employee data stored successfully")
```

```
with open("employee.csv", "r") as f:
```

```
    r = csv.reader(f)
```

```
    for row in r:
```

```
        print(row)
```

4. Explain various file reading methods (**read()**, **readline()**, **readlines()**).

1. Python provides several methods to read data from a file after opening it in read mode. The most common methods are **read()**, **readline()**, and **readlines()**.
2. The **read() method** is used to read the entire content of a file at once. It returns the complete data as a single string.
3. The **read(n)** version of the **read()** method reads only a specified number of characters from the file. This helps when only a small portion of the file needs to be accessed.
4. The **readline() method** reads only one line from the file at a time. It is useful when processing a file line by line.
5. The **readlines() method** reads all lines of a file and stores them in a list. Each line in the file becomes an element in the list.
6. These file reading methods help programmers manage file data efficiently. They allow reading either the entire file, a single line, or multiple lines depending on the requirement.

5. A binary file stores student marks. Write a program to read and display records.

1. Binary files store data in the form of **binary values instead of readable text**. They are commonly used to store data such as images, audio, or structured records.
2. In Python, binary files are opened using modes such as **rb (read binary)** or **wb (write binary)**. These modes allow programs to read or write binary data.
3. To read records from a binary file, the file is opened in **rb mode**. The data is then read using the **read() method**.
4. The program retrieves the stored binary data and displays it on the screen. This helps verify that the binary file content has been read successfully.
5. Example program:

```
f = open("marks.dat", "rb")
```

```
data = f.read()
```

```
print("Student Marks Records:")
```

```
print(data)
```

```
f.close()
```

6. This program opens the binary file, reads the stored records, and displays them. Binary file handling is useful when working with non-text data.

6. Write a program to demonstrate formatted printing using f-strings.

1. **f-strings** are a modern and recommended method for formatting output in Python. They allow variables to be directly inserted into a string.
2. An f-string is created by placing the letter **f** **before the string**. Inside the string, variables are written within curly braces {}.
3. This method makes output formatting simple and readable. It is commonly used for displaying values along with text.
4. f-strings can also format numbers such as limiting decimal places. This helps control how numeric values appear in the output.
5. Example program:

```
name = "Rahul"  
age = 25  
salary = 35000.50
```

```
print(f"Name: {name}")  
print(f"Age: {age}")  
print(f"Salary: {salary:.2f}")
```

6. The program prints the values using formatted output. The salary value is displayed with **two decimal places**, showing how formatting works in f-strings.

(10 – marks)

1. Explain file handling in Python in detail including file operations, modes, and methods.

1. File handling in Python is used to store and retrieve data from files permanently. It allows programs to perform operations such as creating, reading, writing, and updating files.
2. Python uses the **open() function** to open a file before performing any operation. The syntax is `open(filename, mode)` where the mode specifies how the file will be used.
3. Common file modes include **r (read), w (write), a (append), and x (create)**. There are also combined modes like **r+, w+, a+** which allow both reading and writing.
4. Python also supports **binary modes** such as **rb, wb, and ab**. These modes are used when working with non-text files like images, videos, or binary data.
5. After opening a file, different methods are used to perform operations. Some common methods include **read(), readline(), readlines(), write(), and close()**.
6. The **read() method** reads the entire content of the file, while **readline()** reads a single line. The **write() method** is used to store data into a file.
7. After completing file operations, the file must be closed using the **close() method**. Closing the file releases system resources and ensures data is saved properly.

2. Write a detailed program to demonstrate reading and writing text, binary, and CSV files.

1. Python supports different types of file operations such as reading and writing text files, binary files, and CSV files. These operations help store and manage structured or unstructured data. # Writing and reading a text file

```
file = open("textfile.txt", "w")
file.write("Hello Python File Handling")
file.close()
```

```
file = open("textfile.txt", "r")
print(file.read())
file.close()
```

```
# Writing and reading a binary file
file = open("binaryfile.bin", "wb")
file.write(b"Binary Data Example")
file.close()
```

```
file = open("binaryfile.bin", "rb")
print(file.read())
file.close()
```

```
# Writing and reading a CSV file
import csv
```

```
with open("students.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["Name", "Age"])
    writer.writerow(["Rahul", 20])
```

```
with open("students.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

3. In this program, a text file is written and then read using read() method.
4. A binary file is created and read using binary modes wb and rb.
5. A CSV file is created using the **csv module** and data is stored in rows.
6. The csv.reader() function reads each row from the CSV file and prints it.
7. This program demonstrates how Python handles multiple file types effectively.

3. Design a student management system that takes input, stores data in a file, and retrieves it using file handling.

1. A student management system stores student details such as name, roll number, and marks in a file. File handling helps store the information permanently so it can be retrieved later.
2. The program takes input from the user and writes it into a file using the **write or append mode**. This ensures that student records are saved and can be accessed when required.
3. Example program for student management system:

```
# Store student data
```

```
file = open("students.txt", "a")
```

```
name = input("Enter student name: ")
```

```
roll = input("Enter roll number: ")
```

```
marks = input("Enter marks: ")
```

```
file.write(name + "," + roll + "," + marks + "\n")
```

```
file.close()
```

```
# Retrieve student data
```

```
file = open("students.txt", "r")
```

```
print("Student Records:")
```

```
print(file.read())
```

```
file.close()
```

4. In this program, student details entered by the user are stored in **students.txt**. Each record is saved on a new line separated by commas.
5. The program then opens the file in **read mode** to display all stored student records. This allows the user to view previously saved data easily.
6. File handling ensures that the data remains saved even after the program stops running. This makes the system simple but useful for managing student information.

4 . Develop a log file system where data is continuously appended and later analyzed. Implement using file handling.

1. A log file system is used to record events or activities that occur in a program. These logs help track system behavior and analyze problems later.
2. The program uses **append mode (a)** so that new log entries are added without deleting previous data. This ensures that all events are stored continuously in the file.
3. Example program for log file system:

```
# Writing logs
```

```
file = open("logfile.txt", "a")
```

```
event = input("Enter log message: ")
```

```
file.write(event + "\n")
```

```
file.close()
```

```
# Reading logs for analysis
```

```
file = open("logfile.txt", "r")
```

```
print("System Logs:")
```

```
logs = file.readlines()
```

```
for log in logs:
```

```
    print(log)
```

```
file.close()
```

4. In this program, each new log message entered by the user is added to **logfile.txt**. The append mode ensures that previous logs remain unchanged.
5. Later, the file is opened in **read mode** to analyze the stored log messages. The program reads each log entry and displays it on the screen.
6. This type of system is useful for monitoring applications, debugging errors, and maintaining records of system activities.

5. Explain input/output formatting techniques and file handling with suitable examples.

1. Input/output formatting in Python is used to present data in a clear and structured form. It helps display values such as numbers, strings, and variables in a readable format for users.
2. One common technique is the **format() method**, which allows inserting variables into a string using placeholders {}. Example: `print("Name: {} Age: {}".format(name, age))`.
3. Another formatting technique is using **format specifiers** to control the display of numbers. For example, `print("Value: {:.2f}".format(3.4567))` prints the number up to two decimal places.
4. File handling in Python is used to store and retrieve data from files. It uses the **open() function** along with modes such as read (r), write (w), and append (a).
5. Example program demonstrating formatted output and file handling:

```
name = "Rahul"
marks = 89.567

print("Student Name: {}".format(name))
print("Marks: {:.2f}".format(marks))

file = open("data.txt", "w")
file.write("Student: {}\nMarks: {:.2f}".format(name, marks))
file.close()
```

6. In this program, formatted output is used to display values properly. The same formatted data is written into a file using file handling operations.

6. Create a report generation system that reads data from a CSV file and displays formatted output.

1. A report generation system reads structured data from a file and displays it in a formatted way. CSV files are commonly used for storing tabular data such as student records or sales data.
2. Python provides the **csv module** to easily read CSV files and process their rows. Each row in the CSV file is read as a list of values.
3. Example program for generating a report from CSV file:

```
import csv

file = open("students.csv", "r")
reader = csv.reader(file)

print("Student Report")
print("-----")

for row in reader:
    print("Name: {:10} Marks: {:5}".format(row[0], row[1]))

file.close()
```

4. In this program, the CSV file **students.csv** is opened and read using `csv.reader()`. Each row of the file contains student name and marks.
5. The **format() method** is used to display the data in aligned columns for better readability. This makes the output look like a structured report.
6. Such systems are useful for generating reports in applications like student management, business records, and data analysis systems.